

Homework 1

Due: Sept 17th

Instructions

Please read the below instructions carefully before submitting your homework

- Use base R unless stated otherwise.
- When answering in words, use the ‘#’ (comment) option
- Show code for every computational answer.
- Include requested output and brief explanations.
- Use vectorized operations when loops are prohibited.

The total point is 75 points. Each major problem is worth 25 points.

As a submission to the Homework 1, upload the R file in github and include the hyperlink

Problem 1: Objects, data structures, and subsetting

Run the setup code.

```
student_id <- c("S01", "S02", "S03", "S04", "S05", "S06")
section <- c("A", "B", "A", "B", "A", "B")
quiz1 <- c(82, 91, 76, 88, 95, 69)
quiz2 <- c(85, 89, 80, 92, 94, 74)
passed <- c(TRUE, TRUE, TRUE, TRUE, TRUE, FALSE)
```

Part A: Building Data Structures (8 points)

Create the following objects:

- convert `section` to a factor with levels in the order "A", "B";
- create a data frame named `students` containing all five vectors;
- create `score_matrix`, with students as rows, quizzes as columns, student IDs as row names, and `quiz1` and `quiz2` as column names;
- create `course_record`, a list containing `course = "R Programming"`, `scores = students`, and a named vector `cutoffs = c(pass = 70, excellent = 90)`.

Use `typeof()`, `class()`, `length()`, `str()`, and `dim()` where appropriate to inspect these objects. In four or fewer sentences, compare a vector, list, matrix, factor, and data frame.

```
# Your code and answer
```

Part B: Subsetting (8 points)

Using the objects from Part A:

- extract student S04's second quiz score from `score_matrix`;
- extract the first two matrix rows while preserving matrix dimensions;
- extract `course` from `course_record` once with `[`, once with `[[`, and once with `$`;
- briefly explain the difference between `'[`, `'[[` and `'$`.

```
# Your code and answer here
```

Part C: Vectorized calculations (9 points)

Without a loop:

- add `average`, the row-wise mean of the two quizzes, to `students`;
- add `excellent`, equal to `TRUE` when `average >= 90`;
- select section A students whose average is at least 80;
- return only `student_id`, `section`, and `average` for that subset;
- create a named numeric vector of all student averages, using student IDs as names.

Briefly explain why your calculations are vectorized.

```
# Your code and answer here
```

Problem 2: Importing, cleaning, and summarizing data

Run the setup code.

```
csv_text <- "sample_id,site,temp_c,ph,status
M01,North,18.2,7.1,ok
M02,South,20.5,,ok
M03,North,NA,6.8,review
M04,East,22.1,7.4,ok
M05,South,19.7,7.0,review
M06,East,23.0,NA,ok
M07,North,17.8,6.9,ok
M08,South,21.2,7.2,ok"
```

Part A: Importing data (8 points)

Use `read.csv()` with its `text` argument to create `measurements`. Treat both blank fields and the text "NA" as missing.

Then:

- inspect the result with `head()`, `str()`, `dim()`, and `names()`;

- count missing values in each column;
- create `measurements_complete` using `complete.cases()`;
- report the sample IDs removed by the complete-case filter;
- explain why `x == NA` is not a valid missing-value test.

Your code and answer here

Part B: Transform, filter, and summarize (9 points)

Without a loop argument:

- convert `site` and `status` to factors and report their levels;
- create `temp_f = temp_c * 9 / 5 + 32`;
- create `ph_below_7`, preserving missing pH values;
- select complete observations from North or South whose status is "ok";
- return only `sample_id`, `site`, `temp_c`, `temp_f`, and `ph`;
- calculate the overall mean Celsius temperature and the South-site mean, ignoring missing values.

Your code and answer here

Part C: Matrix operations (8 points)

Run:

```
A <- matrix(1:4, nrow = 2)
B <- matrix(5:8, nrow = 2)
```

Compute `A * B` and `A %*% B`. Report the dimensions of both results and explain briefly how element-wise multiplication differs from matrix multiplication.

Your code and answer here

Problem 3: Functions

Run the setup code.

```
student_id <- paste0("P", sprintf("%02d", 1:8))
scores <- c(95, 82, NA, 67, 74, 88, 59, 91)
```

Part A: Write a grading function (9 points)

Write `grade_one()` with default grade boundaries:

```
grade_one <- function(
  score,
  a_min = 90,
  b_min = 80,
  c_min = 70,
  d_min = 60
)
```

Use `if`, `else if`, and `else` to return grades A through F. Return `NA_character_` when the score is missing. Then test the function using the code below, and report its output.

```
grade_one(NA)
grade_one(90)
grade_one(80)
grade_one(85)
grade_one(74)
```

```
# Your code and answer here
```

Part B: Applying functions with loop (8 points)

Create a character vector named `grades` with the same length as `scores`, initially filled with `NA_character_`. Use one `for` loop with `seq_along(scores)` to call `grade_one(scores[i])` and store the result in `grades[i]`. Because `grade_one()` already handles missing values, no separate missing-value condition is needed.

After the loop, assign `student_id` as the names of `grades`, display the completed vector, and explain in one sentence what `i` represents during the loop.

You may begin with:

```
grades <- rep(NA_character_, length(scores))

for (i in seq_along(scores)) {
  # Store one grade here
}

names(grades) <- student_id
```

```
# Your code and answer here
```

Part C-1: Function behavior (4 points)

Write:

```
summarize_scores <- function(x, na.rm = TRUE, digits = 1)
```

It should return a named numeric vector containing total count, missing count, mean, standard deviation, minimum, and maximum. Round the four statistics to `digits` places. Call it once with defaults and once with named arguments.

Then test the function using the code below:

```
summarize_scores( x = scores, na.rm = TRUE, digits = 2 )
```

Part C-2: Function behavior (4 points)

Also write `plot_scores <- function(x, ...)` so that it plots scores against their positions.

Then test the function using the code below:

```
plot_scores(scores, type = "b", pch = 19, xlab = "Position", ylab = "Score", main = "Student Scores" )
```

```
# Your code, predictions, and explanations here
```

Submission checklist

Before submitting, verify that all four problems are complete, every result is supported by code, explanations are concise, required object names are unchanged, and the code can be runnable.